Captured by Tushar Khan

2011 ✗ 1
2012

① Overview of C

*what is program, source program and object program?

Ans: Program: Program refers to the set of instructions which have been writting in a computer language. It tells a computer what to do, how to do and when to do.

Source program: A program written by a programmer in a high level language (Basic, Pascal etc) is called a source program.

Object program: The resulting machine language program is called object program.

* Explain the importance of C language.

Ans: The importance of C language is given below:—

The popularity of C language is increasing day by day due to its many desirable qualities. C is a powerful, flexible, portable and elegantly structured programming language. It is a robust language whose rich set of build in function and operators can be used to write any complex program. It combines the capabilities of an assembly language with the features of high level language. Program written in C efficient and fast. C is highly portable which means that one item of program can be run to another computer. It is well suited for ~~standard~~ structured programming. Another importance of C is its ability to extend itself.

Scanned by CamScanner

2011 * Why c is called a middle level language?

Ans: C is called a middle lavel language which does not mean that c is less powerful or less developed than a high level language (Basic, Pascal etc). The definition of c as a middle level language means that it combines the capabilities of an assembly language with the features of high level language. C allows the manipulation of bits, bytes and address. C code is very portable.
↳ of machine language and asembly language similarly it

For example: If a program written for an apple PC can be moved easily to an IBM PC that program is portable.
↳ Use the data types of high level language

2011 * Differentiate between compiler and an interpreter.

Ans:

Compiler	Interpreter
1) Compiler translates and executes the whole instruction of the source program at a time.	1. An Interpreter translates and executes each instruction of the source program, a ta time
2) Lower execution time.	2) Higher execution time.
3) Debugging is complex and time consuming.	3) Debugging is very easy.
4) Requires higher program development effort and time.	4. Requires less program development effort and time.
5. It translates the whole instruction or program into a machine language 6. A compiler translate	5. It translates one by one or one instruction at a time into a machine language. 7. An interpreter translates in many languages.

Only one language

Scanned by CamScanner

2011 * Describe the basic structure of c ?
2012 * what are the main structural components of c ?
2013 Ans: The main structural components of c is shown in Fig-ⓐ.

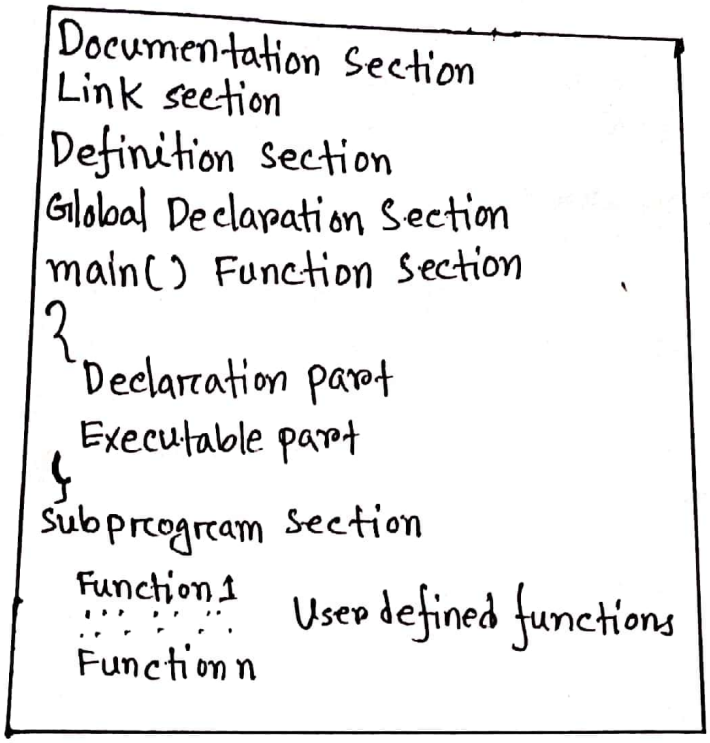


Fig-ⓐ

The main structural components of c program consists of modules called function.

The documentation section consists of a set of comment lines giving the name of the program, the author and other details.

The Link section provides instruction to the compiler.

The definition section defines all symbolic constants.

The global declaration section declares the outside of all the functions. It also declares all the user defined functions.

Every c program must have one main() function section. It contains two part, ⓘ declaration part and ⓘⓘ Executable part.

The programs begins at the opening brace and ends at the closing brace. All statements must end with a semicolon (;).

Captured by Tushar Khan

Scanned by CamScanner

* What is the general form of scanf and printf statement ? OR How can you reading and writting data from keyboard? OR What are the purpose of using printf and scanf function?

Ans: The general form of scanf statement is given below:

scanf("control string", &variable1, ..., &variablen);

The scanf function/statement can be used to reading data from keyboard. The control string indicates specifies the field format in which the data is to be entered and &variable1, &variable2 ....., &variablen specify indicates the address of locations where the data is stored.

The general form of printf statement is given below:—

printf("control string", variable1, variable2, ..., variable n);

The printf function/statement can be used to writting data to output. The control string indicates how many ~~arguments follows and what~~ variables follow and what their types are. variable1, variable2, ..., variablen are formatted and printed according to the specifications of the control string.

Captured by Tushar Khan

Scanned by CamScanner

Constants, Variables and Data Types

2012 2013 CE * What is Keywords in c? What restrictions apply to their use?

Ans: Keywords: Keywords serve as basic building blocks for program statements. All keywords have fixed meaning and these meaning cannot be changed. There are only 32 keywords. They are

auto, break, case, char, const, continue, do, double, default, else, enum, extern, for, float, goto, if, int, long, register, return, short, switch, ~~sing~~ signed, sizeof, static, struct, typedef, union, unsigned, void, volatile, while.

Restrictions: All keywords have fixed meaning and these meaning can not be changed. All keywords must be written in lowercase. The keywords can not be used as a variable name.

2012 CE * What is an identifier? Can you use a keyword as an identifier? Write down the rules for identifier.

Ans: Identifier: Identifiers refer to the names of variables, functions, arrays and pointers. Both uppercase and lowercase characters are permitted in identifiers. These are user defined names and consists of a sequence of letters and digits, with a letter as a first character. Keywords ~~&~~ can not be used as an identifier. Because All keywords have fixed meaning and these meaning can not be changed.

CE The rules for identifiers:

1) 1st character must be an alphabet or (underscore).
2) Must consists only letters, digits or underscore.
3) Only ~~31~~ first 31 characters are significant.
4) Can not use a keyword.
5) Can not contain white space.

Captured by Tushar Khan

Scanned by CamScanner

CE * What is variable? What are the rules for declaring a variable?

2013 Ans: Variable: A variable is a data name that may be used to store a data value. It consists of letters, digits and the underscore character.

for example:
```
int a;
float x;
```

The rules for declaring a variable:

1) They must begin with a letter. Some systems permit underscore (symbol) as the first character.
2) First 31 characters are significant.
3) Uppercase and lowercase are significant.
4) Can not use a keyword.
5) White space is not allowed.

* What is algorithm? Describe the writting process of algorithm.

Ans: Algorithm: An algorithm is the step-by-step procedures for solving a problem.

Writing process of algorithm:

1) 1st ly, start the process.
2) 2nd ly, take the input function.
3) 3rd ly, calculate the function.
4) Then take the result of the function.
5) Finaly, finish the process.

Captured by Tushar Khan

Scanned by CamScanner

2014 (100%) CE

*Describe the fundamental data types in c with its size & range.

Ans: All c compilers support five fundamental data type. They are i) Character type, ii) Integer type, iii) Floting point type, iv) Double type and v) Void type.

Character type: It is used to represent the character type data. It denotes by char. It requires 1 byte or 8 bits. Its range is -128 to 127.

Integer type: It is used to represent the integer type data. It denotes by int. It requires 2 bytes or 16 bits. Its range is -32768 to 32767.

Floting point type: It is used to represent the floating point numbers. It denotes by float. It requires 4 bytes on 32 bits. Its range is $3.4e^{-38}$ to $3.4e^{38}$.

Double type: It is used to represent the real numbers. It denotes by double. It requires 8 bytes on 64 bits. Its range is $1.7e^{-308}$ to $1.7e^{308}$.

Void type: It has no values. It is usually used to specify the type of functions. The type of function is said to be void when it does not return any value to calling functions.

* what is data type?

Ans: Data type: C language rich in its data type. ANSI C supports three classes of data types. They are

i) Primary or fundamental data types
ii) Derived data types
iii) User defined data types

Scanned by CamScanner

* Why we use ~~constant~~ comment in a program?

Ans: A comment is a note that coe put into our source code. In c, the compiler ignores ~~the~~ all comments. ~~It is used primarily to document~~ It gives the meaning and purpose of our source code.

In C, the comment is start by /* and ended by */

For example

/* This is a comment */

* What is tokens?

Ans: Tokens: In a C program, the smallest individual units are known as c tokens. C has six types of tokens. They are

i) Keywords
ii) Identifiers
iii) Operators
iv) constants;
v) strings and
vi) special symbols

* ~~What~~ is constants? How constants can use in c program? Or How can a constant ~~can~~ be declared in a program?

Ans: Constants: Constants refer to the fixed value that do not change during the execution of a program. There are four basic types of constant in C. They are i) Integer constant; ii) floating constant; iii) character constant and iv) String constant.

Integer constant: An integer constant refers to a sequence of digits. For example

-123, 0, 123

Floating or Real constant: The floating constants are represented by numbers containing ~~factorial~~ fractional parts.

For example: 17.325, -7.05, -.008

Captured by Tushar Khan

Scanned by CamScanner

iii) Character constants: A character constant contains a single character enclosed within a single quote marks. For example:

'x', '5'

iv) String constants: A string constant is a sequence of characters enclosed within a double quote marks. For example:

"123", "Hello", "x"

Declaration of a constant: Constant may be declaration/used in two ways. They are given below:—

i) const data_type constant_name = value;

where data_type any standard c data type, constant_name is an identifier and value is the data to be stored.

For example const int x = 30000;

ii) #define constant_name value

where 'constant_name is an identifier and value is the data to be stored.

For example

#define PI 3.1416,

* Explain the terms name, name() and name[]?

Ans: name: Here name is a variable because it is used to store a data value.

name(): Here name() is a function because it declared by a form of function.

name[]: Here name[] is an array because it represented by a form of array.

Scanned by CamScanner

* Can a variable be both constant and volatile?

Ans: we may declare a variable as both constant and volatile as shown below:—

```
volatile const int location =100;
```

see function chapter page(295)

* what are the different storages classes in C?

Ans: There are four types of storage in c. They are given below:—

i) auto: Local variable known only to the function which is declared default to zero.

ii) register: Local variable which is stored in the register.

iii) extern: Global variable known to all functions in the file

iv) static: Local variable which exits and retains its value.

Example:

```
auto in . x;
register char ch;
extern long x;
static in x;
```

* what is the advantage of register storage specifier?

Ans: register is one kind of storage class in C. It is a local variable which is used to stored in the register.

For example register char ch;

Captured by Tushar Khan

Scanned by CamScanner

b

2014 * What are the purpose of qualifiers "const", "void", "volatile", [redacted] ?

Ans: const: Const is used to tell the compiler that the value of the variable must not be changed during the execution of a progream. Ex: const int x=4;

Void: It has no values. It is usually used to specify the type of functions. The type of function is said to be void when it does not return any value to calling functions.

Volatile: Volatile is used to tell the compiler that the value of the variable may be changed at any time by some external sources. Ex: Volatile int x;

(2015) Find out the larger number from two integer numbers using conditional operator.

```
#include <stdio.h>
main()
{ int a, b, max;
  printf("Enter the value of a and b");
  scanf("%d %d", &a, &b);
  max = (a>b)? a:b;
  printf("The maximum number is %d", max);
}
```

For example:

```
const in x=40;

Voltatile int x;
```

Captured by Tushar Khan

Scanned by CamScanner

* How can you declare a variable in a program?

Ans: Declaration of a variable does two things.

i) It tells the complier what the variable name is.
ii) It indicates what type of data the variable will hold.

The declaration of variables must be done before they are used in the program

~~There are two types of declaration. They are~~

i) Primary type declaration: The general form for declaring a variable are given below

data-type v1,v2,.....vn; where data-type refers to the existing data type and v1, v2, ... vn are the names of variables. Variables are seperated by commas. A declaration statement must end with a semicolon. For example:

```
int x;
float y;
double ratio;
```

ii) User defined type declaration: The user defined type declaration can be used to declares variables. It takes the general form:

typedef type identifier;

where type refers to an existing data type and identifier refers to the new name given to the data type. typedef cannot create a new type. some examples are

```
typedef int units;
typedef float marks;
```

Captured by Tushar Khan

Scanned by CamScanner

* Write down the difference between ++i and i++.

Ans: The difference of ++i and i++ is that

1) ++i is pre-increment and i++ is post increment.

2) ++i increment the value first and then return it. On the contray, i++ returns the value first and then increment it.

3) Consider the following statement

```
i = 5;
y = ++i;
```

In this case, the value of y and i would be 6.

when
```
i = 5;
y = i++;
```

In this case, the value of y would be 5 and i would be 6.

* Classify the operator with example?

Ans: Classification of operator: There are eight types of operator. They are given below:—

i) Arithmetic operators. C provides all the basic arithmetic operators. They are given below:—

Arithmetic operator

operator	meaning
+	Addition
−	Sub traction
*	Multiplication
/	Divition
%	Modulo divition

Captured by Tushar Khan

Scanned by CamScanner

Operator & Expression

* what is operator, operands and expression?

Ans: Operator: An operator is a symbol that tells a computer to perform certain mathematical or logical manipulation. Operators are used in programs to manipulate data and variables.

Operands: Operands are the values ~~by which~~ or object of a methamatical operation, or a quantity on which an operation is performed. Consider the expression. $x + y = z$

operands

Expression: An expression is a combination / sequence of operators and operands that reduces to a single value. For example $y = x^2 - 2x + 1$; or $y = x*x - 2*x + 1$)

* what is unary operator?

Ans: Unary operator: Operator act upon on a single operand to produce a new value known as unary operator. Only one operator is associated with the unary operator. For example

i) -5

Alternative: An operator is called a unary operator which consists of only one operand to produce new value is called uninary operator. only one operator is associated with the unary operator.

For example

i) -5

ii) +10

on what is operator? classify them:
operators can be classified into a number of categores

① Arithmetic operators
② Relational operators
③ Logical operators
④ Assignment operators
⑤ Increment and decrement operators
⑥ conditional operators
⑦ Bitwise operators
⑧ Special operators

Scanned by CamScanner

ii) Relational operators: C supports six relational operators in all. They are given below:—

Relational operator

Operator	Meaning
<	less than
>	greater than
<=	less than or equal to
>=	greater than or equal to
==	equal to
!=	not equal to

iii) Logical operators: C supports the following three logical operators. They are given below:—

Logical operator

operator	meaning
&&	Logical AND
||	Logical OR
!	Logical NOT

iv) Bitwise operator: C supports the following bitwise operators.

Bitwise operator

operator	meaning
&	bitwise AND
|	bitwise OR
^	bitwise exclusive OR
<<	shift left
>>	shift Right

They are used for testing the bits or shifting them right or left.

v) Assignment operator: The operator '=' are called assignment operator. It is used to assign the result of an expression to a variable.

Scanned by CamScanner

~~Decition looping~~

vi) Conditional operator: The operator ?: is called conditional operator. The general form of conditional operator is given below:—

```
exp1 ? exp2 : exp3
```

vii) Increment and Decrement operators: The operators ++ and -- are called increment and decrement operators. The operator ++ adds 1 to the operand, while -- subtracts 1. Both taker the following form

```
m++ or ++m
m-- or --m
```

viii) Special operators: C supports the some special operators such as comma operator (,), sizeof operator, pointer operator (* and &) and member selection operators (. and ->).

Captured by Tushar Khan

Scanned by CamScanner

Decision making and looping

C.E * what is statement?

Ans: A statement is a part of program that can be executed.

An expression is a statement. a = a+1;

A function call also a statement. printf("%d", a);

A statement must be end with a semicolon.

* what are the basic decision making statements in C?

Ans: The basic decision making statements in c are given below:—

i) if statement; ii) goto statement; iii) switch statement; iv) conditional operator statement.

These statements control the flow of execution. they are also known as control statements.

* what are the different looping/iterative control statements in C?

Ans: The different looping control statements in c are given below:—

i) for statement; ii) while statement; iii) do-while statement;

~~iv) do-while statement.~~

C.E * Explain if statement with example.

Ans: if statement is a powerful decision making statement and is used to control the flow of execution of statements. The general form of if statement is:—

```
if (condition)
{statement;
}next statement;
```

or

```
if (condition)
{
  statement1;
  statement2;
  . . . . .
  statementn;
}
```

Captured by Tushar Khan

Scanned by CamScanner

The condition have either true or false value. If the condition is true the statement is executed and then goes to the next statement. If the condition is false the control directory goes to the next statement.

Example:

```
#include<stdio.h>
main()
{
int a,b;
printf("a=");
scanf("%d",&a);
printf("b=");
scanf("%d",&b);
if(a>b)
printf("a is greater than b=%d",a);
}
```

* Explain if-else statement with example.

Ans: The if-else statement is the extension of the simple if statement. The general form is :—

```
if (condition)
statement ;
else
statement;
```

where a statement may consists of a single statement / a block of statements or nothing. The else clause is optional.

when the condition is true the if statement / block statements will be executed otherwise else statement / block statement will be executed. In either case either true or false will be executed not both.

```
if (condition)
{ statement 1;
  .....
  statement n;
}
else
{ statement 1;
  .....
  statement n;
}
```

Example:

```
#include<stdio.h>
main()
{ int a,b;
  printf("a=\nb=");
  scanf("%d %d",&a,&b);
if(a>b)
printf("a is greater than b=%d",a);
else
printf("a is less than b=%d",b);
```

Scanned by CamScanner

* Find the largest number among three given numbers using if-else statement.

Ans:

```
#include <stdio.h>
main()
{
int a, b, c;
printf("a=\n b=\n c=");
scanf("%d %d %d", &a, &b, &c);
if (a>b && a>c)
printf("a is largest = %d", a);
else if (b>a && b>c)
Printf("b is largest = %d", b);
else
printf("c is largest = %d", c);
}
```

* Explain if-else-if or else if ladder with example.

Ans: The general form of else if ladder is:—

```
if (condition1)
statement 1;
else if (condition2)
statement2;
else if (condition3)
statement3;
.
.
.
else
statementn;
```

Example:

```
#include <stdio.h>
main()
{
int a,b,c;
Printf("a=\n b=\n c=");
scanf("%d %d %d", &a, &b, &c);
if(a<b && a<c)
Printf("a is minimum=%d",a);
else if(b<a && b<c)
printf("b is minimum=%d", b);
else
printf("c is minimum=%d", c);
}
```

The conditions are executed from the top downward. As soon as any condition is true, the statement is executed and the other statements in the ladder is bypassed (not executed). ~~If none of the conditions are true, the final else is executed.~~ If all other conditional tests fail, the last else statement is executed.

Scanned by CamScanner

Explain nesting if-else statement with example.

Ans: The general form of nesting if-else statement is :—

```
if (condition1)
{ if (condition2)
    statement1;
  else
    statement2;
}
else
{ if (condition3)
    statement3;
  else
    statement4;
}
statementn;
```

Example:

```
#include<stdio.h>
main()
{ int a,b,c;
  printf("a=\n b=\n c=");
  scanf("%d%d%d",&a,&b,&c);
  if (a>c)
  { if (a>b)
      printf("a is maximum =%d",a);
    else
      printf("b is maximum =%d",b);
  }
  else
    printf("c is maximum =%d",c);
}
```

If the condition1 is false, the statement3 will be executed, otherwise it continues to perform the second test. If the condition2 is true, the statement 1 will be evaluated, otherwise the statement2 will be evaluated and then the control is transferred to the statementn.

* Explain the ?: with example ⇒ Ternary Operatore.

Ans: The ?: statement is the alternative form of if-else statement. The general form of ? is:

exp1 ? exp2 : exp3 where exp1 is condition.

The exp1 is executed first. If the result is nonzero, exp2 is executed and returned as the value of the exp1. Otherwise, exp3 is executed and its value is returned.

Example: the segment

```
if (x<0)
  y=0;
else
  y=1;
```

can be written as

```
y = (x<0) ? 0 : 1;
```

Scanned by CamScanner

* Explain switch statement with example.

Ans: The switch statement is the alternative form of if-else-if ~~or Ladder or Lat~~ if else ladder statement. The general form of switch statement is:—

```
switch(expression)
{
  case value1:
        statement1;
        break;
  case value2:
        statement2;
        break;
  . . . . . .
  . . . . . .
  case value n:
        statement n;
        break;
  default:
        statement;
        break;
}
```

For example:

```
#include <stdio.h>
main()
{
  int n, result;
  printf("n=");
  scanf("%d", &n);
  result = n%2;
  switch(result)
  {
    case 0:
      printf("even");
      break;
      case 1:
     printf("odd");
     break;
    default:
     printf("other");
     break;
  }
}
```

The switch statement executed the expression ~~against~~ against a list of case values and when a match is found, this statement ~~associated with that case~~ is executed with that case. exists the loop immediately.

The break statement ~~At the end of each statement signals the end of the particular case~~ and causes ~~an exit from the switch statement, transferring the control to the statement~~

The default is an optional case. When present, it will be executed if the value of the expression does not match with any of the case values.

Captured by Tushar Khan

Scanned by CamScanner

* Explain for loop with example.

Ans: The for loop is an entry-controlled loop that provides a more concise loop control structure. The general form of for loop is:—

```
for(initialization ; condition ; iteration)
{
    statement block;
}
```

The for loop is used to repeat a statement or block of statements for a specified number of times.

The execution steps of for statement/loop are given below:—

i) The initialization is an assignment statement that is used to set up the loop control variable.

ii) The condition is a relational expression that determines when the loop exits. If the condition is true, the statement is executed, otherwise the loop is terminated.

iii) The iteration defines how the loop control variable change each time the loop is executed.

The three major seetions seperate by semicolons.

Example: print the numbers 1 through 100 on the terminal.

```
#include<stdio.h>
main()
{
    int i;
    for(i=1; i<=100; i++)
    printf("%d", i);
}
```

Captured by Tushar Khan

Scanned by CamScanner

* Explain while statement with example.

Ans: The while loop is an entry controlled loop statement. The general form of while loop is :—

```
while(condition)
{
  statement block;
}
```

or

```
while (condition)
{statement 1;
 statement 2;
 statement n;}
```

Where the statement is either an empty statement, a single statement or a block of statements. The condition ~~is any~~ may be any expression and true is any nonzero value. The loop repeats a statement while the condition is true. If the condition becomes false, the control is transferred out of the loop.

Example:

```
#include<stdio.h>
main()
{int n, i=0, sum=0;
scanf("%d", &n);
while (i<n)
{sum=sum+i;
  i++;
}
printf("%d", sum);
}
```

For example:—

```
#include<stdio.h>
main()
{int n;
do printf("Enter the value n=");
{scanf("%d", &n);
}
while (n>0);
getch();
}
```

```
#include<stdio.h>
main()
{int n;
do
{printf("n=");
 scanf("%d", &n);
 n=n++; or n=n+1;
}
while(n<=10)
}
```

2012 * Explain do-while statement with example.

Ans: The general form of do-while loop is:—

```
do
{statement;
}
while (condition);
```

The do-while ~~so~~ loop checks its condition at the bottom of the loop. This means that this loop always executes at least once. If the condition is true, the loop will repeat. The do-while loop repeats/iterates until the condition becomes false. The condition must be Boolean expression.

Scanned by CamScanner

2014 * Explain goto statement with example.

Ans: The general form of goto statement is

```
goto label;
.....
other statement(s);
label:
.....
```

The goto statement will jump to any point of program.

where label is any valid either before or after goto. Each labeled statement within the program must have a unique label i.e no two statements can have the same label.

Example:

```
if (n==0)
goto error;
else
n+=n;
error:
printf("\n Error! value should not be zero");
```

2014 * Explain break statement with example.

The break command exits the loop immediately.

Ans: The break statement has two use. It is used to terminate a case in the switch statement and another used to ~~force immediate termination of a loop~~ terminate a loop, bypassing the normal loop conditional test. The general form of break statement is:-

```
break;
```

When the break statement is encountered inside a loop, the loop is immediately terminated and program control goes ~~resumes at~~ the next statement following the loop.

For example:

```
#include<stdio.h>
main()
{int i;
for(i=0; i<100; i++)
{ printf("%d", i);
  if (i==10)
  { break;
  }
return 0;
```

```
#include<stdio.h>
#include<math.h>
main()
{ int n; while(1)
 { printf("n=");
   scanf("%d", &n);
   if (n<=0)
   break;
   printf("%d\n\n", sqrt(n));
 }
getch();
}
```

Scanned by CamScanner

* Explain continue statement with example.

Ans: The general form of continue statement is

```
continue;
```

The continue statement is used to bypass the remainder of the current pass through a loop. The loop does not terminate when a continue statement is encountered. The continue statement can be included within a while, a do-while or a for statement.

In while and do-while loops, program control goto the conditional tests. In for loop, control go to the iteration position and then goto the conditional expression.

For example:

```
#include <stdio.h>
main()
{
 int n;
 printf("\n Even number");
 for (n = 1, n<=100 ; n++)
 { if (n%2)
   continue;
   printf("%d", n);
 }
 printf ("\n\n\n odd number\n");
  n = 0;
 while(n<100)
 {
  n++;
  if (!(n%2))
   continue;
   printf ("%d", n);
 }
 getch();
}
```

```
#include <stdio.h>
main()
 { int n,i;
 printf("n=");
 scanf("%d", &n);
 printf("Even number");
 for(i = 0; i <n ; i++)
 { if (n%2)
   continue;
   printf("%d", n);
 }getch();
 }
```

Scanned by CamScanner

* Differentiate between if and if-else statement.

Ans:

if statement	if-else statement
1) General form:— if (condition) {statement; next statement; if(condition) {statement 1; statement n; }	1) General form: if(condition) statement 1; else statement 2; if (condition) {statement 1; statement n; }else {statement 1; statement n; }
2) If the condition is true statement and next statement are executed in sequence.	2. If the contidition ~~true~~ is true, ~~statement~~ statement 1 is executed.
3. If the condition is false, next statement is executed.	3. If the condition is false, statement 2 is executed.

(Q11) Ans: * Differentiate between while and do-while statement.

~~Ans:~~ while statement	do-while statement

1) General form:—
while (condition)
{
statement;
}
	1) General form:—
	do
	{
	statement;
	}
	while (condition)

2. Entry controlled loop statement.	2. Exit controlled loop statement.
3. Condition is executed at the top of the loop.	3. Condition is executed at the bottom of the loop.
4. If the condition is false, statement does not execute.	~~4. If the condition~~ 4. Statement is always executed at least ~~once~~ one.
5. The condition may be any expression and true is any nonzero value.	5. The condition must be Boolean function.

Captured by Tushar Khan

Scanned by CamScanner

* Differentiate between switch and if-else statement.

Ans:

switch case statement	if-else statement

1) General form:
switch (expression)
{
case value1:
statement-1 ;
break;
. . . . . .
case value n:
statement-n ;
break;
default:
statement;
break;
}
	1) General form:
	if (condition)
	{statement-1;
	. . . . . .
	statement-n;
	}
	else
	{statement-1;
	. . . . . .
	statement-n;
	}

2) switch can run much faster.	2) If-else can run slowly.
3) It can test only the equality.	3) It can test any types of Boolean expression.
4) Break is needed.	4) Break is not need.
5) More efficient way to represent multiple branching.	5) Less efficient way to represent multiple branching.
6. No two case constant on the switch can have identical value.	6. There is no restriction in if statement.
7. It can not use a range of value easily.	7. It can use a range of value easily.

Captured by Tushar Khan

Scanned by CamScanner

* what are the conditional / selection control statements in C?

Ans: The conditional statements in C are given below:

i) If statement;
ii) Switch-case statement;
iii) The ternary operator ?:

* what is infinite loop?

Ans: Infinite loop: An infinite loop is a piece of coding that lacks a functional exit so that it repeats.

For example: In for loop:

```
for(;;)
```

In while loop:

```
while(1)
```

* what are the jump statements in c?

Ans: The jump statements in C are given below:—

i) break; ii) continue; iii) goto; iv) return;

* what are the label statements in c?

The label statements in C are given below:—

i) case and ii) default

* what is the return statement?

Ans: Return statement: Return statement is used for returning a value from function definition to calling function. The general form of return statement is

```
return (expression);
```

for example:

```
return 0;
return a;
return (a+b);
```

Captured by Tushar Khan

Scanned by CamScanner

Array

CE * What is array & static arrays & Dynamic Array?

Ans: Array : An array is a collection of ~~element~~ same data type elements with a fixed size. It can be used to represent a list of numbers or a lists of names.

static arrays: The process of allocating memory at compile time is known as static memory allocation and the arrays ~~that~~ received at compile time are called static array.

Dynamic arrays: The process of allocating memory at run time is known as dynamic memory allocation and the arrays created at run time are called dynamic array. It is used to creating and manipulating data structure such as linked lists, stacks & queues.

CE * Write down the advantage of array.

Ans: The advantage of array are given below:—

i) Arrays as member of structures
ii) Arrays as dynamic data structures
iii) manipulating character arrays and strings.
iv) Using pointers for accessing arrays
v) using structure type data as array element
vi) Passing arrays ~~of~~ as function parameters.

Captured by Tushar Khan

Scanned by CamScanner

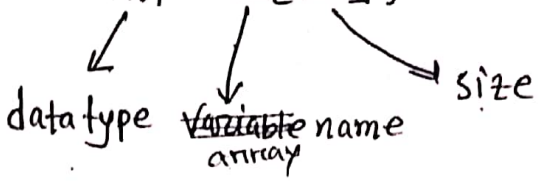
Q/* How to declare and initialize a one dimensional array.

Ans: The general form of one dimentional array declaration is

data-type ~~variable_name~~ array_name[size];

For example

int x[100];



Initialization of one dimentional array:

An array can be initialized at either of the following stages:—

i) At compile time:— The general form of initialization of an array is

data-type ~~variable~~ array_name[size] = { list of values };

The values in the list are seperated by commas. For example: int x[3] = {0, 1, 2}; OR

char name[6] = {'B', 'A', 'K', 'U', 'L', '\0'};

or char name[] = "Bakul";

ii) At run time:— An array can be initialized at run time. This approach is ~~usually applied~~ used for initializing large arrays. For example

```
int x[3];
scanf("%d%d%d", &x[0], &x[1], &x[2]);
```

OR

```
for(i=1; i<=100; i++)
  a[i] = i;
```

```
int i,n,x[10];
for(i=0;i<n;i++)
scanf("%d", &x[i]);
```

Scanned by CamScanner

CB * How to declare a two dimensional & ~~three~~ multi dimensional array?

Ans: Declaration of two ~~dy~~ dimensional array are given below:—

```
data_type array_name[row_size][column_size];
```

For example: int x[2][3];

Declaration of multidimentional array are given below:—

```
data_type array_name [size][size][size];
```

For example: int x[2][3][4];

* How to initialized a two dimentional array?

Ans: The initialized a two dimentional array are given below:—

Like the one dimentional array, two dimentional array may be initialized by following their declaration with a list of initial values enclosed in braces.

For example: int x[2][3] = {0,0,0,1,1,1};

This statement also written as

int x[2][3] = {{0,0,0},{1,1,1}};

We can also initialize a two ~~dy~~ dimentional array in the form of a matrix as shown below:—

```
int x[2][3] = {
                {0,0,0},
                {1,1,1}
              };
```

Captured by Tushar Khan

Scanned by CamScanner

* What is searching & sorting?

Ans: Searching: Searching is the process of finding the location of the specified element in a list. The most commonly used search techniques are

i) Binary search and
ii) Sequential search / Linear search.

Sorting: Sorting is the process of arranging elements in the list according to their values, in ascending or descending order. The most commonly used sorting techniques are

i) Bubble sort;
ii) selection sort;
iii) Insertion sort;

* what is the difference between variable and array?

Ans: Difference between variable & array :—

Variable	Array
1) A variable is a data name that may be used to store a data value.	1) An array is a collection of same data type elements with a fixed size.
2) It consists of letters, digits and underscore character.	2) It represents a list of numbers or a list of names.
3) For example: int x;	3) For example: int x[100];
4) General form: data-type variable-name;	4) General form: data-type array-name[size];

Scanned by CamScanner

Character Array & string:

* What is string?

Ans: String; A string is a sequence of characters that represented by arrays of characters. Any group of characters defined between double quotation marks is a string constant. For example:

"Man is not free from error"

or

printf ("well done!");

* Write down the common operation performed on character string.

Ans: The character string are often used to build meaningful and readable programs. The common operation performed on character string are given below:—

i) Reading & writing strings,
ii) Combining strings together,
iii) Comparing strings for equality,
iv) Copying one string to another,
v) Extracting a portion of a string.

* How to declare and initialize a string variable?

Ans: A string variable is any valid c variable name and is always declared as an array of character. The general form of declaration of a string variable is ~~char~~ ~~string~~ char string_name[size];

For example char students[50];

The end of the string is marked with a special character, the null character ('\0').

P.T.O

Scanned by CamScanner

C also permits us to initialize a character array (without specifying the number of elements.) In such cases, the size of the array will be determined automatically, based on the number of elements initialized. For example

```
char string[4] = {'a', 'b', 'c'};
char city[9] = "Bakul"; char string[4] = "abc";
```

* What is the difference between 'A' & "A" ?

Ans: Here 'A' is a character type data and "A" is a string. In a program

```
char ch= 'A';  ——→ 1 byte
char ch[] = "A"; ——→ 2 byte
```

* what is the input function of a string?

Ans: The input function of a string are given below:—

i) scanf ii) getchar iii) gets

scanf function: The familar input function scanf can be used with %s format specification to read in a string of characters from keyboard. In this case, the ampersand(&) is not required before the variable name.

For example:

```
char address[10];
scanf ("%s", address);
```

The scanf function can be used to reading a line of text. such as

```
scanf("%5s", st);
scanf("[^\n]", st);
puts(st);
```

P.T.O

Scanned by CamScanner

getchar function: This function is used repeatedly to read successive single characters from the input and place them into a character array. It takes the following form

```
char ~~int~~ ch;
ch = getchar();
```

The getchar function has no parameter.

gets function: Another and more convenient method of reading a string of text containing white space is to use the library function gets(); it is ~~a simple~~ one parameter function ~~function with one string parameter~~ and it takes the following form:

```
gets(st);
```

* what is the output function of string?

Ans: The output function of string are given below:

i) printf ; ii) putchar ; iii) puts .

printf function: The familar output function printf can be used with %s formate specification to print string to output screen. For example

```
printf("%s", st);
```

It can also specify the precision with which the array is displayed for instance. the specification

```
printf("%10.4s", st);
```

It also allows for variable field width or precision for instance. printf("%*.*s\n", w, d, st);

For example :-

```
char st[] = "FICE";
printf("%3.*s", d, st);
```

Scanned by CamScanner

putchar function: It is a one parameter function and takes the following form

```
char ch = "A";
putchar(ch);
```

~~The putchar function requires one parameter~~

This statement is equivalent to

```
printf("%c", ch);
```

puts function: It is a one parameter function and takes the following form:—

```
puts(st);
```

It display a line of text.

* Briefly explain the string handaling functions.
OR List the string related standard library function with their general form.

Ans: The most commonly used string handling functions are given below:—

i) strcat(): The strcat function is used to joins two strings. It takes the following form

strcat(str1, str2); where str1 and str2 are character array.

ii) strcpy(): The strcpy function is used to copies one string to another. It takes the following form

strcpy(str1, str2);

iii) strcmp(): The strcmp function is used to compares two string. It takes the following form

strcmp(str1, str2);

iv) strlen(): The strlen function is used to find the length of a string. It counts and returns the number of characters in a string. It takes the following form

n = strlen(str); where n is an integer variable.

Scanned by CamScanner

Pointers

* What is pointers?

Ans: Pointers: A pointer is a variable that represents the location of the data item, such as a variable or an array element.

For example, it can be used to pass information back and forth between a function and its reference point.

* Why pointers are used in C program?

Ans: Pointers are used in the following case :—

i) Pointers are more efficient in handling arrays & data ~~&~~ tables.

ii) It can be used to return multiple values from a function via function arguments.

iii) It can be used to support dynamic memory management.

iv) It reduces ~~code~~ length & complexity of programs.

v) It increases the execution speed & reduces the program execution time.

vi) It provides an efficient tool for manipulating dynamic data structure.

vii) Pointer are flexible. We can make the same pointer to point to different data variables in different statements.

Captured by Tushar Khan

Scanned by CamScanner

* How to declare a pointer and initialize it?

Ans: The declaration of a pointer variable takes the following form:—

```
data_type *pointer_name;
```

This tells the compiler three things that

i) The * tells that the pointer_name is a variable.

ii) pointer_name needs a memory location

iii) pointer_name points to a variable of data_type.

Initialization of pointer variable;

Pointer initialization is the process of assigning address of a variable to pointer variable. It contains address of variable of same data type. The address operator & is used to determine the address of a variable.

When a pointer variable has been declared we can use the assignment operator to initialize the variable. For example:

```
int x;
int *p;    /* declaration */
p = &x;    /* initialization */
```

The above expression can also combine the initialization with the declaration. i.e

```
int *p = &x;
```

Captured by Tushar Khan

Scanned by CamScanner

* What are the pointer operators in c?

Ans: There are two types of pointer operators in c. They are * and &.

The * operator is used to define pointer variables and to dereference a pointer.

The & operator is used to determine the address of a variable.

Captured by Tushar Khan

Scanned by CamScanner

* What is NULL in a pointer?

Ans: NULL is a special value which may be assigned to a pointer. It indicates that this pointer does not point to any variable. For example

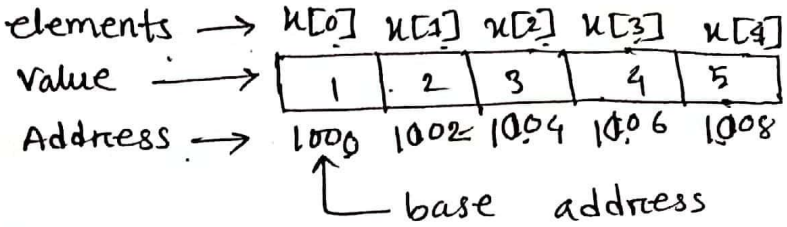
```
int *X = NULL ;
```

* Explain how a pointer can be generated to an array?

Ans: Let we declare an array x as follows

```
int x[5] = {1,2,3,4,5};
```

suppose the base address of x is 1000. Hence we have



Suppose Then base address of x is 1000, the location where x[0] is stored. That is

```
x = &x[0] = 1000;
```

If we declare p as an integer pointer, then the pointer p to point to the array x by the following statement `p = x;`

This is equivalent to `p = &x[0];`

When handling arrays, instead of using array indexing, we can use pointers to access array elements.

Captured by Tushar Khan

Scanned by CamScanner

* How to pass a pointer to a function ?

Ans: A pointer to a function is declared as follows

type (*fptr) ();

This tells the compiler that fptr is a pointer to a function, which returns type value. The parenthesis around *fptr are necessary.

see the function chapter

We can make a function pointer to point to a specific function by simply assigning the name of the function to the pointer. For example

```
double x(int, int);
double (*p1)();
p1 = x;
```

The statement declares p1 as a pointer to a function and x is a function and then make p1 to point to the function x.

* Define call by reference & call by value.
OR Explain call by reference technique with example.

Ans: Call by reference: The process of calling a function using pointers to pass the address of variables is known as call by reference.

Call by value: The process of passing the actual value of variables is known as call by value.

Captured by Tushar Khan

P.T.O.

Scanned by CamScanner

Call by reference with example:

consider the following code

```
main()
    { int x;
      x = 20;
      f(&x);
      printf("%d", x);
    }
    f(int *p)
    {
      *p = *p + 10;
    }
```

When the function f() is called, the address of variable x not its value, is passed into the function f(). Inside the function f(), the p is pointer variable ~~variable p is declared as a pointer~~ and therefore the pointer p is the address of variable x.

```
*p = *P + 10;
```

since p represents the address of x, the value of x is changed from 20 to 30. Hence, the output of the program will be 30 not 20.

* How is a variable accessed through its pointer?

Ans: When a pointer variable has bee declared, the indirectional operator * is used to access the value of the variable by using the pointer. For example: int x, n, *P;

```
x = 10;
P = &x;
n = *P;
```

In this example, the value of n would be 10 since p is the address of x. The statements p = &x; and n = *P; are equivalent to n = *x; which in turn is equivalent to n = x.

Scanned by CamScanner

/ Function /

* What is function? Write down the importance of a function.

Ans: Function: A function is a self contained block of code that performs a particular task. Every function must have a main function to indicate where the function has to begin its execution.

The importance of function are given below:—

i) It is easier to understand, debug, and test of a program.

ii) It is easy to locate and isolate a faulty function for further investigations.

iii) It reduces the length and complexity of a program. ~~The length of a source program can be reduced by using functions at appropriate place.~~

iv) It facilitates top down modular programming

v) It is used by many other programs.

* Write down the general form of function declaration

Ans: The general form of function declaration are given below:—

```
function_type function_name(parameter list)
{
body of the function
}
```

*What is user defined function? what is the basic difference between library function and user defined function.

Ans: User defined function: A user defined function has to be developed by the user at the time of writting a program. However, it can later become apart of the the c program library. In fact, this is one of the strengths of c language. The basic difference is that library functions are not required to be written by us whereas a user defined function has to be developed at writting a program.

Scanned by CamScanner

✱ What are the elements of a user defined function? explain it?
OR: what is function definition, function call, function declaration,

Ans: Elements of user defined function:

There are three elements of a function. They are
i) Function definition;
ii) " declaration &
iii) " call.

Function ~~declaration~~: definition: The function definition is an independent program module that is specially written to implement the requirements of the function. It is also known as function implementation.
It takes the following elements:
i) Function name; ii) Function type; iii) Parameter list;
iv) Local variable declarations; v) Function statements and
vi) Return statement.
All six elements are grouped into two parts. They are
i) Function header and ii) Function body

Function header: It consists of three parts and take the following form:

function_type function_name (parameter_list)

Function body: It consists of three parts and takes ~~the~~ like as following example:

```
int sum(int a, int b)
{
    int sum;
    sum = a+b;
    return(sum);
}
```

Scanned by CamScanner

Function declaration: A function declaration also known as function prototype consists of four parts. They are i) Function type; ii) Function name; iii) Parameter list and iv) terminating semicolon.

The general format of function declaration is

function_type function_name (parameter_list);

It is similar to the function header except the terminating semicolon.

For example: int sum(int x, int y);

Function call: A function call ~~can be used~~ are can be used in a function name followed by a list of actual parameters or arguments, if any enclosed in parenthesis.

For example:

```
main()
{ int x;
  x = sum(10, 5)
  printf("%d", x);
}
```

OR: A function call is a postfix expression which contains a list of actual parameters or arguments if any enclosed in parenthesis.

* what is actual parameter & formal parameter?

Ans: Actual parameter: The parameters used in function calls are called actual parameter. It is used in a calling statement may be simple constants, variables or expressions.

Formal parameter: The parameters used in prototypes and function definitions are called formal parameter.

* what is function arguments / parameters?

Ans: Function arguments: Argument refers to the data this is passed to function. Function accepts argument and it return a value back to the calling program. For example int max(int x, int y)

A function can be called with or without ...

Scanned by CamScanner

*Differenciate between local variable and global variable:

Ans: The difference between local variable and global variable are given below:—

Local variable	Global variable
1) Local variables are those variable ~~which~~ which are defined inside a ~~program~~ function	1) Global variable are those variable ~~which~~ are defined outside a ~~program~~ function.
2) They are known only ~~to~~ the block in which it is defined.	2) They are known throughout the program.
3) Storage for local variable is on the stack.	3) storage for global variable in the fixed region of memory set.

* What are the different classes storage in c ?

Ans: There are four different classes storage in C. They are

i) Automatic variable: Automatic variables are declared inside a function in which they are to be utilized. They are created when the function is called and destroyed automatically when the function is exited. They are also known as local or internal variable. For example: auto int x;

For example

```
main()
{
  auto int number;
  .....
  .....
}
```

ii) External variable: External variable are declared outside a function in which they are active and alive throughout the ~~enter~~ entire program. They are also known as global variable. For example: extern int x;

```
int x;
main()
{x=10;
.....
}function() { int x=0;
```

Scanned by CamScanner

Static variable: static variable may be declared inside a function or outside a function inwhich they are initialized only once, when the program is compiled. Internal static variable are those which are declared inside a function. Also an external static variable are those which are declared outside a function. They are also known as global variable. For example:-

```
static int x;
static float y;
```

Register variable: Register variables are local variable which is stored in a register. Register will lead to faster execution of a program. for example

```
register char ch;
```

* what do you mean scope, visibility & lifetime ?

Ans: Scope: The region of a variable in which a variable is available for use is called scope.

visibility: The visibility refers to the accessibility of a variable from the memory.

Lifetime/Longevity: The lifetime of a variable refers to the duration of time in which a variable exists in the memory during execution.

* what are the techniques for passing array to a function?

Ans: There are three techniques for passing array to a function.

They are

i) Formal parameters as a pointer ⇒

```
void function(int *x)
{
.....
}
```

ii) " " " " sized array ⇒

```
void function(int x[50])
{
...
}
```

iii) " " " unsized array ⇒

```
void function(int x[])
{
.....
}
```

Scanned by CamScanner

* How to pass an array to functions?

Ans: Passing array to functions are given below:—

The name of the array is passed as a parameter or an argument/

i) The function must be called by passing only the name of the array, parameter (i.e starting address of memory area is passed as argument).

ii) In the function definition, the parameter must be an argument/ array type, without any subscripts and the size of the array as arguments/parameters.

iii) The function prototype must show that the argument/ parameter is an array.

For example, the call

largest(a,n)

will pass the whole array a to the called function.

The largest function header might look like:

float largest(float array[], int size)

* How to pass an ~~array to~~ pointer to functions?

Ans: Passing pointer to functions are given below:—

i) The type of actual & formal arguments must be same.

ii) The actual arguments must be the addresses of variable that are local to the calling function.

iii) The formal arguments must be prefixed by the indirection operator *.

iv) In the prototype, the arguments must be prefixed by the symbol *.

v) The access the value of an actual argument in the called function, we must use the corresponding formal arguments prefixed with the indirection operator *.

For example:

```
void function(int *x)
{
```

Scanned by CamScanner

* How to pass a string to functions?

Ans: Passing string to functions are given below:-

i) The string to be passed must be declared as a formal argument of the function when it is defined. For example:

```
void display(char name[])
{
.....
}
```

ii) The function prototype must show that the argument is a string.

```
void display(char str[]);
```

iii) A call to the function must have a string array name without subscripts as its actual arguments.

```
display(names);
```

where names is a properly declared string array in the calling function

* What is recursion? explain with example.

Ans: Recursion: A function that calls itself is known as recursive function. It is The technique is known as recursion in c programming. It is very useful to solve many mathematical problems such as calculating the factorial of a number, generating fibonacci series etc.

For example:

```
Void main()
{
printf("This is an recursion example\n");
main();
}
```

When executed, it produces the following outputs

```
This is an recursion example
" " " " "
" " " " "
This is an recursion
```

Execution is terminated abruptly, otherwise the execution will continue indefinitely.

Scanned by CamScanner

* what is category of a function?

Ans: The catagory of a function are given below:

i) Function with no arguments and no return values.
ii) " " " " but return a value.
iii) " " arguments and no return values
iv) " " " " one return value.
v) " that return multiple values.

* Explain function with no aregument and no return value with example.

Ans: Function with no arguments and no return value

When a function has no arguments, it does not receive any data from the calling function. similarly, when it does not return a value, the calling function does not receive any data from the called function. In this case, there is no data transfer between the calling function and called function.

For example:

```
#include <stdio.h>
void area();
void main()
{
  area();
}
void area()
{ float r, area;
  printf("r=");
  scanf("%f", &r);
  area = 3.1416*r*r;
  printf("Area = %f", area);
}
```

Captured by Tushar Khan

Scanned by CamScanner

* Explain function with argument and return value with an example.

Ans: Function of argument & return value:

When a function has arguments, it can receive any data from the calling function. Similarly, when it returns a value, the calling function receives any data from the called function. In this case, there is a data transfer communication between the calling function and the called function.

For example:

```
#include <stdio.h>                    float area(float r);
float area(float r);
main()                                float area(float r)
{                                     { float A;
  float r, Area;                        A = 3.1416*r*r;
  printf("r=");                       return(A);
  scanf("%f", &r);                    }
  Area = area(r);
  printf("\n %f", Area);
}
```

* Explain function with argument but no return value with an example

Ans: Function with argument but no return value:

When a function has arguments, it can receive any data from the calling function. Similarly, when it does not return any value, the calling function does not receive any data from the called function.

For example:

```
#include<stdio.h>                     void area(float r)
void area(float r);                   {float A;
main()                                 A = 3.1416*r*r;
{ float r, Area;                       printf("Area=%f", A);
  printf("r=");                       }
  scanf("%f", &r);
  area(r);
}
```

Scanned by CamScanner

*Explain function no arguments but return value with example

Ans: Function with no argument but return value:

When a function has no arguments, it does not receive any data from the calling function. Similarly, when it returns a value, the calling function receives any data from the called function. For example:—

```
#include <stdio.h>
float area(float r);
void main()
{ float m;
  m = area();
  printf("%f", m);
} float area(float r)
  { float A;
    A = 3.1416*r*r;
    return(A);
  }
```

* Argument & Return value

```
#include<stdio.h>
int main()
{int a,b,c;
printf("Enter the value of a & b");
scanf("%d %d", &a, &b);
c=sum(a,b)
printf("%d", c);
}int sum(int a, int b)
{ int c;
  c=a+b;
  return c;
}
```

*No argument & No return value

```
#include<stdio.h>
void main()
{sum();
}
void sum();
{int a,b,c;
 printf("a=\n b=");
 scanf("%d%d", &a,&b);
 c=a+b;
 printf("%d", c);
}
```

* No argument but return value

```
#include<stdio.h>
void main()
{int c;
  c=sum();
 printf("%d", c);
}
 void sum()
 { int a,b,c;
   printf("a=\n b=");
   scanf("%d%d", &a, &b);
   c=a+b;
  return c;
 }
```

* Argument but no return value

```
#include<stdio.h>
  int main()
  { int a,b;
    printf("a=\n b=");
    scanf("%d%d", &a, &b);
  sum(a,b);
  }int sum(int a, int b)
   { int c;
     c=a+b;
    printf("%d", c);
```

Scanned by CamScanner

/ Structure & Union */*

* What is structure?

Ans: Structure: A structure is a collection of variable of different types under a single name for better handling. The keyword struct is used for creating structure. Also it helps to organize complex data in a more meaningful way. Example of structure are

Time: Seconds, Minites, Hours

Date: Day, Month, Year

* How can you decare a structure?

Ans: A structure variable declaration is similar to the declaration of variables of any other data type. It includes the following elements:—

i) The keyword struct
ii) The structure tag name
iii) List of variable names seperated by commas
iv) A terminating semicolon.

The general form of declaration a structure variable are given below:—

```
struct [structure tag]
{
    data_type name1;
    data_type name2;
    .  .  .  .  .  .
    .  .  .  .  .  .
    data_type name;
} [one or more structure variables];
```

captured by Tushar Khan

Scanned by CamScanner

* Explain with an example, how member variables of a structure can be accessed?

Ans: There are two operators used for accessing members of a structure. They are

i) Member operator (.)
ii) Structure pointer operator (->)

Any member of a structure can be accessed as

structure_variable_name.member_name

Suppose, we want to access salary for variable p1 then it can be accessed as: p1.salary

For example; book->price
book.price

is the variable representing the price of book.

we can also use printf or scanf to give the values as

```
printf("%d", book.price);
scanf("%d", &book.price);
printf("%d", book->price);
scanf("%d", &book->price);
```

* Differentiate between arrays & structures.

Ans: The difference between array & structure are given below:-

Array	structure
1) An array is a collection of same data type elements with a fixed size.	1) A structure is a collection of variable of different data type under a single name for better handling.
2) An array is derived data type.	2) A structure is a programmer defined one.
3) No keyword is used to creat an array.	3) struct keyword is used to creat a structure.
4) It is used to represent a list of numbers or a list of names.	4) It is used to represent a set of attributes such as student_name, student_rool and marks.

5) For example
int x[50];
	5) For example:
	struct telephone{
	int number;
	char name;
	};

Scanned by CamScanner

* what is union ? Differenciate between structure & union.

Ans: Union: A union is a special data type available in c that can be used to store different type of data in the same memory location. The keyword union is used for creating union variable.

The difference between structure & union are given :—

Structure	Union
1) Definion :—	1) Definition :—
2) The keyword struct is used for creating structure variable.	2) The keyword union is used for creating union variable.
3) In structure, each member have its own storage location.	3) In union, All members have same memory location.
4) The general format of structure is struct [structure tag] { member definition1; member definition2; . . . . . member definition; } [one or more structure variable];	4) The general format of union is union [union tag] { member definition1; member definition2; . . . . . member definition; } [one or more union variables];
5) Example: struct Data { int x; float y; } data;	5) Example: union Data { int x; float y; } data;

* How many ways to create custom data types?

Ans: There are five ways to create custom data types. They are

i) The structure
ii) The union
iii) The typedef
iv) The bit field
v) The enumeration.

Scanned by CamScanner

* Explain with an example, how member variables of a union can be accessed?

Ans: The member access operator(.) is used for accessing variables to a union.

Any member of a union can be accessed as

union_variable_name.member_name

For example; code.m
code.x
code.c

For example:

```
#include<stdio.h>
union Data{
        int x;
        float y;
};
main()
{ union Data value;
  value.x = 10;
  value.y = 5.5;
  printf("%d", value.x);
  printf("%f", value.y);
}
```

```
#include<stdio.h>
#include<string.h>
union Data{ int i;
            float f;
            char str[20];
};
int main()
{ union Data data;
  data.i = 10;
  data.f = 220.5;
  strcpy(data.str, "C programming");
  printf("%d", data.i);
  printf("%f", data.f);
  printf("%s", data.str);
  return 0;
}
```

* What is sizeof structure?

Ans: Size of structure: The unary operator sizeof is used which tell us the size of a structure or any variable. The expression of size of structure is

sizeof(struct x)

If y is a simple structure variable of type struct x, then the expression sizeof(y)

captured by Tushar Khan

Scanned by CamScanner

* what is typedef structure?

Ans: typedef structure: Programmer generally use typedef while using structure in c language. Also ~~typedef use~~ typedef is used to creat custom data type variables.

For example:

```
typedef struct complex
{int x;
 float y;
} comp;
```

* what is Bit fields?

Ans: Bit fields: A bit field is a set of adjacent bits whose size can be from 1 to 16 bits in length. The name and size of bit fields are defined using a structure. The general form of bit fields are given below.

```
struct [tage_name]
{ member definition 1: bit_length;
  member definition2: bit_length;
  .  .  .  .  .  .  .  .  .  .
  member definition : bit_length;
};
```

For example:

```
struct max
{int x: 2;
 int count: 5;
};
```

captured by Tushar Khan

Scanned by CamScanner

* Explain array of structure?

Ans: Array of structure: An array of structures of each element of the array representing a structure variable. For example

~~struct marks~~ struct class student[100];

An array of structure is stored inside the memory in the same way as a multidimentional array.

* Explain structure within structure?

Ans: Structure within structure: Structure within a structure means nesting of structures. It is permited inc.

For example:

```
struct student
{ char name;
  char department;
  struct
  { int roll;
    float GPA;
  };
};
```

```
struct complex{
    int x;
    float f;
};

struct number{
    struct complex c;
    int y;
} n1, n2;
```

To access x for n2 structure variable, structure member n1.c.x is used.

* How do you use structure as function parameter?

Ans: C suppors to pass structure values as a function argument or parameter. There are three methods by which the values of a structure can be transferred from one function to another.

First method: By this method, ~~pass~~ each member of the structure is passed ~~values as argument to function~~ as an actual argument of the function call.

Second method: By this method, passing of a copy of the entire structure to the called function.

Third method: ~~This method involves a concept called pointer to pass the structures as an arguments~~. By this case, the address location of the structure is passed to the called functi

The general format of sending a copy of a structure to the called function is function_name(structure_variable_name);

The called function takes the following form:

```
data_type function_name(struct_type st_name)
{ ..... :
  return(expression);
}
```

Scanned by CamScanner

* How can you pass a pointer to structure?

Ans: We can define pointer to structures in the same way as the pointer to any other variable. We can store the address of a structure variable above defined - pointer variable.

For example:

```
struct book{
    char title[50];
    char author[50];
    char subject[100];
    int book id;
};
```

```
struct book *struct_pointer;
struct book book1;
struct_pointer = &book1;
struct_pointer ->title;
```

* How to initialize a structure variable?

Ans: The initialization of structure variable are given below:—
The structure variable can be initialize in the compile time.
It takes the following ~~forms~~ elements:—

i) The keyword struct
ii) The structure tag name
iii) The assignment operator (=)
iv) The ~~name of the~~ variable name to be declared
v) A set of values for the members of the structure variable seperated by commas and enclosed in braces.
vi) A terminating semicolon (;)

Now, The initialization of structure variable must be done only in the declaration of the actual variables. ~~There~~ The following statement initialize two structure variables. Here, it is essential to use a tag name.

```
main()
{
    struct record
    { int roll;
      float gpa;
    };
    struct record student1 = {29, 3.90};
    struct record student2 = {45, 3.75};
}
```

captured by Tushar Khan

Scanned by CamScanner